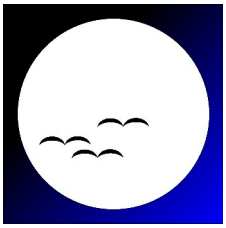

QSpice C-Block Components

Sharing Data Between Component DLLs

Revision Date: 2025.12.10

Caveats & Cautions

First and foremost: Do not assume that anything I say is true until you test it yourself. In particular, I have no insider information about QSpice. I've not seen the source code, don't pretend to understand all of the intricacies of the simulator, or, well, anything. Trust me at your own risk.

If I'm wrong, please let me know. If you think this is valuable information, let me know. (As Dave Plummer, ex-Microsoft developer and YouTube celebrity (Dave's Garage) says, "I'm just in this for the likes and subs.")

Note: This is the tenth document in a clearly unplanned series:

- ***The first C-Block Basics document covered how the QSpice DLL interface works.***
- ***The second C-Block Basics document demonstrated techniques to manage multiple schematic component instances, multi-step simulations, and shared resources.***
- ***The third C-Block Basics document built on the second to demonstrate a technique to execute component code at the beginning and end of simulations.***
- ***The fourth C-Block Basics document revisited the Trunc() function and provided a basic "canonical" use example.***
- ***The fifth C-Block Basics document revisited the MaxExtStepSize() function with a focus on generating reliable clock signals from within the component.***
- ***The sixth C-Block Basics document examined connecting and using QSpice bus wires with components.***
- ***The seventh C-Block Basics document described recent QSpice changes (a new Display() function changes to Trunc()).***
- ***The eighth C-Block Basics document detailed post-processing techniques and much more in painful detail.***
- ***The ninth C-Block Basics document explained how to use the QSpice EngAtof() function.***
- ***The tenth C-Block Basics document was a technical reference for the QSpice "Berkeley Sockets API."***
- ***The eleventh C-Block Basics document extended the "Berkeley Sockets API" framework to overcome the limitations of template-generated code.***
- ***The twelfth C-Block Basics document created multi-client "Berkeley Sockets" host servers for C++, Python, and Java.***

This paper explores exchanging data directly between DLLs. It assumes that you have followed along with the earlier Berkeley Sockets API papers. See my GitHub repository here for prior documents in this series.

Cautionary Note

I suppose that it's fitting that this is the 13th C-Block Basics paper – "13" being considered unlucky in some cultures. If you use the cleverness outlined in this paper carelessly, it will bring you no end of frustration….

The below describes an "advanced technique." You really must have a good understanding of how C-Block components work – the QSpice simulation cycle, the evaluation function and, if used, `MaxExtStepSize()` and `Trunc()` functions – before the below will make any sense. If you don't already understand the prior papers in this series, well, you shouldn't tread here….

Overview

In this paper, I'll share a technique to share data directly between two component DLLs. That is, we'll have two distinct DLLs and one will fetch data from the other.

I'll show you how to do it and then try to demonstrate/explain why you shouldn't.

How To Share DLL Data

The example code for this paper includes two schematics and two DLLs:

CB13_Test_A.qsch, CB13_Test_B.qsch	Example schematics, identical except for netlist order (see below).
CB13_MyDLL1.cpp (X1)	Exposes data that CB13_MyDLL2.dll can fetch data from this DLL.
CB13_MyDLL2.cpp (X2)	Fetches data exposed in CB13_MyDLL1.dll with external linkage.

Observe that X2 is dependent on X1. That is, X2 is fetching data from X1 – X1 must be updated by QSpice before X2 is evaluated at each simulation time-point.

So, let's take a look at the code.

MyDLL1 declares several variables like this:

```
extern "C" __declspec(dllexport) double sharedTimeStep = 0.0;
extern "C" __declspec(dllexport) double sharedInVal    = 0.0;
extern "C" __declspec(dllexport) double sharedOutVal   = 0.0;
```

The `extern "C" __declspec(dllexport)` prefix makes these global variables available for dynamic linking. The values are updated in the evaluation function. Of particular importance, `sharedTimeStep` is set to the current time-step parameter ("`t`").

MyDLL2 does the following to access the MyDLL1 variables:

```
HINSTANCE handle      = LoadLibrary("cb13_mydll1.dll");
inst->pSharedTimeStep = (double *)GetProcAddress(handle, "sharedTimeStep");
inst->pSharedInVal    = (double *)GetProcAddress(handle, "sharedInVal");
inst->pSharedOutVal   = (double *)GetProcAddress(handle, "sharedOutVal");
```

This happens in the "per-instance initialization" portion of the code. `LoadLibrary()` is called to get a handle to the MyDLL1 DLL instance. It then retrieves and saves pointers to the MyDLL1 global data

variables in per-instance data. If any of the returned pointers are null, there was a failure and the code displays a message and terminates the simulation.

The important value is the current simulation time-step value ("`t`") last saved to MyDLL1's global `sharedTimeStep` variable. If this isn't the same as the time-step in MyDLL2 ("`t`"), we have a problem and a message is displayed.

The Problem

To see the problem in action, open CB13_Test_A.qsch and run the simulation. (Don't edit this schematic.) It should display "`X2: MYDLL2 WARNING: sharedTimeStep=0 != t=2e-13 -- Check netlist!`" in the QSpice Output window.

Then open CB13_Test_B.qsch and run the simulation. (Again, don't edit the schematic.) It should run without the error message.

Compare the waveforms from both simulations. The plot of `V(TDiff)*1s/1V` tells you the difference in the last time-step processed in MyDLL1 vs MyDLL2. It should be a flat line at 0s if MyDLL1 has been evaluated before MyDLL2 is called. Otherwise, MyDLL2 was evaluated before MyDLL1 and the "dependency" has been broken.

The schematics appear to be identical. What's the difference?

Well, open the schematic netlists. The order of the components has changed. All I did was save CB13_Test_A.qsch to CB13_Test_B.qsch, cut X1 (CTRL-X), paste it elsewhere in the schematic, and move it back into position. This simple editing of the schematic changed the netlist order. And that was enough to change the order of evaluation of the components.

This is a problem. A BIG problem IMHO. If you plan to share your components, they should meet professional standards:

- Adding/deleting/moving components should not change the simulation behavior. This example fails because it is dependent upon the DLL evaluation order.[1]
- Having multiple instances of a given component (or components) should not break anything. That generally means keeping all instance data in per-instance data and not using global variables. If you duplicate X1 & X2 in the example schematic, you'll mess up the netlist order. The example may or may not detect the issue as it uses only a time-step value to detect dependencies.

My advice: Don't use shared DLL global variables unless you absolutely must do so. If you must, then write code to detect corrupted evaluation order. For a single component instance (X2) that is dependent upon a single upstream component (X1), the example code may work. If you allow multiple instances of the components, well, you'll need to take additional steps to ensure that the code doesn't break without warning the user.

---

1 I confirmed with Mike Engelhardt that, as of this writing, component DLLs are called in reverse netlist order. However, this is not a guaranteed evaluation order in future releases. Code defensively!

Summary

If you are careful and the only user of your project, you can indeed share data between DLLs. Include the time-step test to ensure that you don't screw up the evaluation order.

If you are creating projects to share with others, well, they will probably break your carefully constructed netlist and call you long past your bedtime. I suppose that it might be a good thing if you charge by the hour….

Acknowledgments

Thanks to Mike Engelhardt, the author of QSpice, for suggesting the techniques discussed in this document albeit for a rather different and safer purpose.

Conclusion

I hope that you find the information useful. If you implement any of the enhancements, please do share with the community.

And, of course, let me know if you find problems or suggestions for improving the code or this documentation.

--robert